University of Toronto
Faculty of Applied Science and Engineering
MIE335 Project Report


| |
|---|
| 1.0 Introduction |
| 2.0 Algorithms (Pseudo codes and Complexity analysis) |
| 2.1 Brute Force |
| 2.2 Rectangle Division |
| 2.3 Supernal |
| 3.0 Instance Generation |
| 4.0 Algorithm Comparison |
| 5.0 Memory Issue |
| 6.0 Conclusion |

## 1.0 Introduction

In this project, the team is required to solve a Multidimensional 0-1 Knapsack Problem using three different methods: Brute Force Method, Rectangle Division Method, Supernal Method and develop an optimal solution.

Brute-Force method aims to find all the feasible solutions to the 0-1 Knapsack problem. In this method all of the J objective images of the feasible solutions are calculated and stored. Duplicated objective images will be removed and the Nondominated images are found by removing the dominated objective points.

### 1.1 Rectangle Division Method vs Brute Force Method

The principle of the Rectangle Division Method is to divide the criterion space into smaller regions in order to search for NDPs more efficiently and minimize the effect of other points. By narrowing down the coverage of NDPs, the Rectangle Division Method will check less regions (rectangles in this case) compared to Brute Force Method. Also, the Rectangle Division Method is less complex than the Brute Force Method which requires less time and effort to find the NDPs by conducting a complexity analysis. Using complexity analysis, the team gains an insight about how long the code will run for the largest test cases that are used to test our program's correctness.

### 1.2 Supernal Method vs Brute Force Method

Similar to the rectangular method, the supernal method conducts a search of NDPs in the criterion space by dividing it into smaller regions. Each region is denoted by its northeast point, i.e., the point with the largest values in all dimensions. By converting the MOP into a weighted-sum single objective optimization problem, we could utilize the Gurobi package to search for NDPs much more efficiently than the brute force method which checks all possible feasible points in the decision space.

## 2.0 Algorithms (Pseudo codes and Complexity analysis)

In this section, the team introduces three methods for solving the Multiobjective Multidimensional 0-1 Knapsack Problem. The algorithms, improvement and complexity analysis are developed in details.

### 2.1 Brute Force

Brute Force method requires finding all the feasible solutions and calculates their objective images prior to comparing their objective images. In order to find the feasible solutions, one approach is to reverse engineer the optimization by first creating a solution out of $2^n$ possibilities and then check its feasibility. The feasibility of a given solution is checked by iteration through all the constraints. If a solution meets all the constraints, it is feasible.

**Brute Force function- Initial**

Every item out of n items is either in the sack or not, let 1 and 0 represent the item being included and not included, respectively. Create every combination of items being in the sack by producing 0 and 1 ‘s in a list of size n, and store all the created possibilities (points) in the list named lst. [“product” method in the “itertool package” is used to create all the 2^n possibilities.] For each possible combination of items in the sack check if it is feasible. In other words, iterate through each of the constraints in b, and check if it is violated. For every feasible solution, each of the objective images are calculated by the dot product of the objective coefficient vector and the point. The vector of objective images is created.

```
import itertools
import numpy as np
Def BruteForce(n,b,c,a,J):
    NDP=[] # Initialize NDPs
    Z_s=[] # Initialize a list to store the objective vectors (z1,z2,..,zj)
```

```
    lst = list(itertools.product([0, 1], repeat=n))
    for each point ∈ lst do
      check=true # Define a variable check as the assumption that point is feasible
      for k in range(len(b)) do
              If a[k].point> b[k] then # Dot product to calculate lhs. for constraint kth constraint

# If a constraint is violated, stop checking the rest of the constraint by breaking from the loop.
                check=false
                break

# If a combination of items in the sack is feasible, calculate the value of each objective function in c.
       if check then
            Z=[]   # Initialize a vector for objective images for each feasible point
            for each z ∈ c do    # for each objective coef vector in c
                Append z.point to Z # Dot product to calculate each objective image and append the
objective image to the vector Z
            if Z not in Z_s then # if the objective image does not already exist in Z_s
                Append Z to Z_s
```

**Remove Dominated Images**

The ultimate goal is to find the non dominated points of the criterion space in the maximization problem. However, to align with other methods and output format, we have transformed maximization to minimization problem by introducing negative objective coefficients. Therefore, here we remove the points **with largest images (least negative)** as they represent the dominated points in the maximization problem.

```
    NDP=Z_s[:]   # create a copy of all feasible images and put them in NDP
    already_removed=[ ] # Initialize an array to keep the dominated regions as they are removed
    for point in range(0,len(Z_s)) do
        if Z_s[point] not in already_removed then
            for p2 in range(point+1,len(Z_s)) do   # for all the images vectors appearing after point
in Z_s list
# First assume point (A) is dominated by p2 (B) in the maximization problem
                check_remA = true
                check_remB = false
                ind = -1
                if Z_s[p2] not in already_removed then
# Iterate through all each objective images in point(A)and p2(B)
                    for j in range(J) do
# if the jth objective image of p2 is larger (least negative) compared to jth objective image in point
                        if Z_s[point][j]<Z_s[p2][j] then
                            check_remA=false # the assumption of point being dominated in
maximization problem is no longer true
                            ind=j #store the index j in variable ind
                            Break #stop checking the rest of the objective images
                    if ind==0 then # if the first objective image in point was least negative than of p2, it
is possible that p2 is dominated by point in the maximization problem
                        check_remB=true
                        for j in range(1,J) do
# if there is an objective image in point that is larger (least negative) compared to corresponding objective image in
p2, it is no longer possible for p2 to be dominated by point
                            if Z_s[point][j]>Z_s[p2][j] then
                                check_remB=false
                                Break
                    if check_remA then
                        Remove Z_s[point]from NDP
                        Break #stop checking point being dominated by next p2s
                    else if check_remB then
                        Remove Z_s[p2]from NDP
                        Append Z_s[p2]to already_removed

    return NDP,len(NDP),0
```

**Brute Force Initial Algorithm - Complexity analysis**

itertools.product([0, 1], repeat=n) has a complexity of $n^n$ based on the python documentation [1]
Lst list contains $2^n$ points, each point has n dimensions.
The outermost for loop iterates through all the elements in list, which are in total $2^n$ iterations.
In the worst scenario, all the $2^n$ solutions are feasible. Let the elementary operation be the **dot product of constraints coefficient vector a_k and the point**, as this operation has the highest complexity and repeated the most. The dot product elementary operation is represented with D in the analysis.
$o(D) = N * Multiplications + N - 1 * Additions$
Assume o(Addition)=o(Multiplication)=constant for sake of simplicity. o(D)=(2N-1)*Additions = (2N-1)O(1)=2N-1

In the first iteration of the outermost loop, In the worst case scenario, the elementary operation is executed for all of the k constraints. Additionally, given that the point is feasible, for each of j objectives, the dot product of objective coefficient vector c_j and the point is executed which is the same as the elementary operation. For each feasible point it is checked whether the objective image already exists in Z_s, which has the complexity O(N), N being the size of the Z_s. Note that in iteration i, Z_s has i-1 elements before append operation.
$complexity\ in\ iteration\ ith\ of\ feasible\ point:\ k * o(D) + J * o(D) + O(i-1) = (k+J) * (2N-1) + O(i-1)$
In total of $2^n$ iterations, in the worst case all iterations are feasible points therefore the complexity is as follows.
$n^n + 2^n * [(k+J) * (2n-1)] + O(0) + O(1) + .. + O(2^n) = n^n + 2^n * (2n(k+j)) + \frac{2^n*(2^n-1)}{2}$

**Complexity analysis of Remove Dominated part**

Remove Dominated points, in the worst case scenario none of the points are dominated by each other. Let the elementary operation be the comparison of jth objective image in the if statement in the third nested loop (**"`if` `Z_s[point][j]<Z_s[p2][j]` `then`"**), as it is the most recurring operation in the nested loops. Let the complexity be represented by C. Note that comparison takes constant time, O(C)=1.
First iteration of the outermost for loop:
At worst case $2^n - 1$ points are checked. For each of the $2^n - 1$ points, "**`if`** `Z_s[point] not in already_removed`" is checked. Searching for an object in a list in python has complexity of O(n), n being size of the list, which is in worst case 0 as no point is dominated.

After, the If statement is evaluated as true, J Comparison operations are done. (**"`if` `Z_s[point][j]<Z_s[p2][j]` `then`"**) is evaluated as true for the jth objective image and the point is not to be removed. In general for the ith iteration of the outermost for loop complexity is in the worst case (N-i)*(J)C
In total the complexity is
$(2^n - 1) * (J)C + (2^n - 2) * (J)C + ... + (1) * (J)C = \frac{(J)(2^n-1)(2^n)}{2} C$

The remove dominated is in total : $\frac{(J)(2^n-1)(2^n)}{2} C$

**Total complexity of BruteForce initial:**

$$O(2^n * (2n(k+j)) + \frac{2^n*(2^n-1)}{2} + \frac{(J)(2^n-1)(2^n)}{2} O(c)) \approx O(n^n + (n) * 2^n + 4^n) = O(n^n)$$

**Brute Force Algorithm - Improvement**
The Removed Dominated part of the initial algorithm is improved by first sorting the points in the criterion space. Z_s is sorted by descending order, sorting places the point with the least negative objective value

of z1, at the beginning of the list. In fact, sorting ensures that for a given point A in the list, there is **no** point B appearing after point A for which the relation B dominated by A holds in the maximization problem. Therefore, we only need to check if point A is dominated by any of the points Bs that appear after A.

```
#--removing dominated part for the improved algorithm----
  Sort Z_s in desc order
  NDP=Z_s[:] # Create a copy of Z_s in NDP
  For point in range(len(Z_s)) do   #Iterate through the points in Z_s
        j=point+1 # let j be the index of next point
        while(j<=len(Z_s)-1) do # while not passing the last point
            check=true #First Assume point is dominated by the next point_j in the maximization problem
            for z in range(1,J) do #go to all objective images z2,z3,..,zJ
                if Z_s[point][z]<Z_s[j][z] then # if the jth objective image of point_j is larger (least negative) compared to jth objective image in point
                    check=false #point can no longer be dominated by point_j thus, stop checking the rest of the objective images
                    break
#If the assumption of point being dominated by point_j was true, remove the point from list NDP and stop checking the next point_js by breaking from the while loop.
            if check then
                Remove Z_s[point] from NDP
                break
            else then j+=1   # otherwise add 1 to index j and continue checking A being dominated by point_j+1.

    return NDP,NDP.length,0
```

**Complexity analysis - Improved Brute Force**

Sorting Z_s : N logN where N is size of the criterion space at worst case being $2^n$

Let the elementary operation be the comparison of jth objective image in the if statement in the third nested loop (**"if** `Z_s[point][z]<Z_s[j][z]` **then"**), as it is the most recurring operation in the nested loops. Let the complexity be represented by C. Note that comparison takes constant time, O(C)=1.

First iteration of the outermost for loop:

At worst case $2^n - 1$ points are checked. For each of the $2^n - 1$ points, J-1 Comparison operations are done, in the worst case. No point is dominated by any point in the worst case, therefore, "`Remove Z_s[point] from NDP`" is not operated.

In general for the ith iteration of the outermost for loop complexity is in the worst case

$(2^n - i) * (J-1) * o(C)$

In total the complexity is

$$(2^n - 1) * (J-1)C \ + \ (2^n - 2) * (J-1)C \ + ... + (1) * (J-1)C \ = \frac{(J-1)(2^n-1)(2^n)}{2} o(C)$$

The remove dominated is in total : $2^n log\ 2^n \ + \ \frac{(J-1)(2^n-1)(2^n)}{2} O(C)$

Note that the complexity of the improved Brute Force algorithm is

$$O(n^n \ + 2^n * (2n(k+j)) + \frac{2^n*(2^n-1)}{2} + 2^n log\ 2^n + \frac{(J)(2^n-1)(2^n)}{2} O(c)) \approx O(n^n \ + (nlog2 \ + n) * 2^n + 4^n) = O(n^n)$$

It seems that the improved version is worse, as the both algorithms include many break statements and identifying the real worst case is challenging. Additionally, the assumptions for the worst case are not true, as there are a limited number of feasible solutions considering the design of Generate Instance function. As a result, average case complexity analysis is more reasonable for this problem and could justify the better performance of the improved Bruteforce according to the results of testing.

### 2.2 Rectangle and Division Method

- Pseudo code and explanation

Rectangle division method aims to solve problems with 2 objectives. The rectangle is the form of the search regions, and we use two points to represent a rectangle. NDPs are found by searching the optimal points in the rectangles using the gurobi model. The main function called Rectangular(MOP) is based on the algorithm in the project description. The other two functions called Solver(MOP,p) and Solverplus(MOP,r,p,e) utilize gurobi models to achieve the function that finds the optimal points.

```
Def Rectangular(MOP):
      FoundNDPs = [] # Initialize FoundNDPs
# Firstly, find the 2 corner points:  z_nw has minimal z1, and then minimal z2; z_se has minimal z2, and then
minimal z1; add them into the FoundNDPs.
      p1 = [2,1] #represents priority, optimize z1 first
      p2 = [1,2] #represents priority, optimize z2 first
      Solve lex min(z1,z2) by Solver(MOP,p), get optimal solution = z^NW
      FoundNDPs.add(z^NW)
      Solve lex min(z2,z1) by Solver(MOP,p), get optimal solution = z^SE
      FoundNDPs.add(z^SE)
      Rectangles = {R[z^NW , z^SE]} # Define the initial Rectangles
      RecCount = 1    # Define the number of regions
      while Rectangles is not empty do
          R[z^1,z^2] ∈ Rectangles
          Rectangles.Remove(R[z^1,z^2])
          mid = (z_2^1 + z_2^2)/2  # This is the midline of the Rectangles
          R2 = R[(z_1^1,mid), z^2]
          RecCount += 1
# Get optimal point zval in R2
          Solve lex min(z1,z2) by Solverplus(MOP,r,p,e), get optimal solution = zval
          if zval ≠ z^2 and zval ≠ 'NA' then
               FoundNDPs.add(zval)
               Rectangles.add(R[zval,z^2])
          R3 = R[z^1, (zval-e,mid)]
          RecCount += 1
          Solve lex min(z2,z1) by Solverplus(MOP,r,p,e),get optimal solution = zval1
# Get optimal point zval in R3
          if zval1 ≠ z^1 and zval1 ≠ 'NA' then
               FoundNDPs.add(zval1]
               Rectangles.add(R[z^1,zval1])
      Sort FoundNDPs
      Remove repetitive points in FoundNDPs
      return FoundNDPs, FoundNDPs.length, RecCount
```

```
# This function is used to only find z_nw and z_se through the gurobi model.
  The input p refers to priority of optimization.
Def Solver(MOP,p):
    model -> gurobi model
    var -> model.addBinaryVariables with size n
    for i in range b.length do
        Constraint -> sum(var[i]*a[j][i] for i in range n)<= b[j]
        Add Constraint to model
# There are 2 objectives,when lex min(z1,z2),optimize z1 first then z2; when lex min(z2,z1), optimize z2 first then z1
    for i in range J do            # J = 2, i = 0,1
        Obj -> sum(var[k]*c[i][k] for k in range n)
        model.set_MultiObjective(obj, index=i, priority = p[i], GRB.MINIMIZE)
   Update model
   Optimize model
# Get the optimal point
   opt0 -> model.get_Objective with index = 0
   opt1 -> model.get_Objective with index = 1
   zval=[opt0.value,opt1.value]
```

```
    return zval
```

```
# This function is used to find other NDP points through the gurobi model.
   The inputs: r refers to R2 or R3; p refers to priority; and e refers to epsilon, a very small number.
Def SolverPlus(MOP,r,p,e):
    model-> Gurobi model
    var -> model.addBinaryVariables with size n
    for i in range b.length do
        Constraint -> {sum(var[i]*a[j][i] for i in range n)<= b[j]},
# The 4 constraints below define the rectangle's region
    Constraints ->[ {Upper bound <= r[0][1] + e},{Left bound >= r[0][0] - e},
                    {Lower bound >= r[1][1] + e},{Right bound <= r[1][0] + e} ]
    Add all constraints to model
# There are 2 objectives,when lex min(z1,z2),optimize z1 first then z2; when lex min(z2,z1), optimize z2 first then z1
    for i in range J do          # J = 2, i = 0,1
        Obj -> sum(var[k]*c[i][k] for k in range n)
        model.set_MultiObjective(obj, index=i, priority = p[i], GRB.MINIMIZE)
   Update model
   Optimize model
# Get the optimal point
   opt0 -> model.get_Objective with index = 0
   opt1 -> model.get_Objective with index = 1
   zval=[opt0.value,opt1.value]
   return zval
# For the case that no point found in a rectangle
   if no point found in the rectangle region do
     return 'NA'
```

**Improvements -- Check if the point is dominated**

Description: For each iteration, there might be the optimal points with the same z1 value generated in the rectangle region, to avoid dominated points and repetitive regions, we did a little changes on the Rectangular function and create a new function called 'contain':

```
Def Rectangular(MOP):
   …
  while Rectangles is not empty do
          R[z^1,z^2] ∈ Rectangles
          Rectangles.Remove(R[z^1,z^2])
          mid = (z_2^1 + z_2^2)/2  # This is the midline of the Rectangles
          R2 = R[(z_1^1,mid), z^2 ]
          RecCount += 1
# Get optimal point zval in R2
          Solve lex min(z1,z2) by Solverplus(MOP,r,p,e), get optimal solution = zval
         if z2 == None do     # changes, if there is no z2
           continue
          if zval ≠ z^2 and zval ≠ 'NA' then
              if zval not contain in FoundNDPs then # changes,call function 'contain' here
                FoundNDPs.add(zval)
                Rectangles.add(R[zval, z^2 ])
         R3 = R[z^1, (zval-e,mid)]
         RecCount += 1
         Solve lex min(z2,z1) by Solverplus(MOP,r,p,e),get optimal solution = zval1
          if z1 == None do # changes, if there is no z1
            continue
          if zval1 ≠ z^1 and zval1 ≠ 'NA' then
                FoundNDPs.add(zval1]
                Rectangles.add(R[z^1,zval1]))

#-------------------------
# This is the function used to check if there is a dominated points
def contain(p, ndp, obj):
```

```
# checks if the FoundNDPs list contains the same z1 value for optimal point in R2
    if p == p1 then
        for i in ndp do
            if obj[0] == i[0] then
                return True
    return False
```

**Rectangle Division method - Complexity Analysis**

We consider the worst case. Let n be the input size of variables. Then the number of generated possible feasible points(excluding z=0) is $2^n - 1$.

1. Gurobi IP model complexity $\rightarrow 2$
   We define 1 gurobi model as 1 elementary operation, for each while loop iteration, there are 2 gurobi models used, therefore, there are 2 operations for each iteration.
2. Division of regions $\rightarrow 2^{n-1}$
   For each iteration, there will be $2^{n-1}$ regions generated. For example, the initial rectangle splits to 2 rectangles, then at the end of the 1st iteration, there are 2 rectangles R2 and R3 in the same level waiting for the next 2 iterations: R2 divides to 2 rectangles, R3 also divides to 2 rectangles, which becomes 4 rectangles at the next same level.
3. Number of iteration $\rightarrow 2^{n-1} - 1$
   The depths of iteration depends on the while loop. For example, let n = 3, then there would be maximum 8 points generated, for the worst case, there will be 7 optimal points need to be searched, then there will be iterating 3 times, for the 1st time, 4 optimal points are found, the 2nd iteration, 1 or 2 points are found in one of the 2 divided rectangles; in the 3rd iteration, 1 or 2 points are found in the another division. Similarly, when n = 4, 16 points can be generated and needs to have 7 iterations.

In conclusion, $Complexity\ for\ the\ worst\ case: (2^{n-1} - 1) * 2^{n-1} + 2 * 2^{n-1} = 2^{2n-2} - 2^{n-1} + 2^n \rightarrow O(4^n)$

### 2.3 Supernal Method

- Pseudo code and explanation

Supernal method aims to solve general MOP problems. Theoretically, it should be able to solve problems with any $j \geq 2$ objectives. Since the objective function coefficients are all non-positive and we want to minimize each objective function, the initial region is always denoted by its northeast(NE) corner point: the origin, i.e., $(0,0,0)$ for $j = 3$ cases.

```
Def supernal(MOP):
    FoundNDPs = [] #initialize an empty list
    #initialize the region and define the supernal point of the MOP as the origin
    Region = [[0.0]*J]
    #convert the MOP into a weighted-sum single objective optimization problem (SOP)
    λ_j = 1 for all j  #randomly pick 1 as the weight of each objective
    for k in range(J):#sum up λ_j z_j for all j =1,...,J
    obj_coef = np.add(obj_coef,np.multiply(lambdas,c[k]))
    #initialize explored regions counter
    regCount=0
    while Regions is not empty do:
      Pick the first region of Regions, R[z^NE] to explore
      regCount += 1 #increment region counter
      Solve min{λ^T z : z ∈ R[z^NE]} #solve the SOP
   #if the status of the gurobi model is 2, a NDP is found
          if feasible then
          z* = Newly found NDP
          FoundNDPs.add(z*)
          for each R[z̄] in Regions do
            if z* is included in this R[z̄] then Regions.remove(R[z̄])
```

```
        for j=1,...J do #split into J new regions
          make a copy of z* : z^new ,where z^new_j = z*_j − 1
          Regions.append(z^new)
      else: Regions.remove(R[z^NE]) #if no NDP is found in this region so remove current region
      #Check region dominance
      if J ≥ 3 then
        Reg = Regions[:]
        for r1 in Regions[:]:
          for r2 in Regions[:]:
            Dom = True
            if r1 != r2:
              for j in range(J):
                if r1[j] > r2[j]:#r1 is not dominated by r2
                  Dom = False
            if Dom then Reg.remove(r1)#r1 is dominated by r2, remove r1
        Regions = Reg

    Return FoundNDPs, FoundNDPs.length, RecCount
```

```
Improvement - Check region dominance:

      if J ≥ 3 then
      #check and remove dominated regions for MOP with more than 2 objectives
      Regions.sort()#sort the regions ascendingly based on the Regions[0] dimension
      Reg = Regions[:] #make a copy of current sorted regions
      for pt in range(len(Regions))do
        j -> pt+1 # Reg[j] is the next region after Reg[point]
        while(j <= len(Regions)-1)do
          check=True
          for z in range(1,J) do
          # Regions is already sorted, don't need to compare j = 1 dimension
            if Reg[pt][z] > Reg[j][z] then
              check=False #Reg[pt] is not dominated by Reg[j]
              break # don't need to check the rest of the dimensions
              if check then
                Reg.remove(Reg[pt]) # dominated by Reg[j]
                break # move on to check the next region
              else: j += 1 #compare Reg[pt] against the next following region
      Regions = Reg #only non-dominated regions remain
```

**Supernal method - Complexity Analysis**

We consider the worst case: let n be the input size of decision variables. Then the number of potential feasible points(excluding z=0) is $2^n - 1$. Here we assume the decision space and criterion space have one-to-one mapping, meaning there is $2^n - 1$ NDPs to explore.

Case 1: General case (J = 2)

Major activities in the supernal method are:

1) Solve a Gurobi IP model
   The supernal method solves a Gurobi IP model in each iteration to search for a NDP. We here define an elementary operation as solving a Gurobi model. The complexity for this part is thus 1.
2) Remove explored regions
   After a new NDP is found, we check to remove regions that include this NDP. In the worse case, we can only remove the region we just processed. The complexity for this part is thus 1.
3) Branch out new regions
   By observation, the number of new regions to branch out is related to the current number of iterations. In the first iteration, a total of **J** new regions are generated. For the following iterations, the number of new regions generated is: $J * (J-1) * (i-1)$, where i denotes the $i^{th}$ iteration. We define the complexity to create a new region as 1.

Therefore, the *time complexity* for the worst case is: $(2^n - 1) * (1 + 1) + J^2(J - 1) * \sum_{i=2}^{2^n-1} (i - 1)$

$O(2^n)$

Case 2: J $\geq$ 3

For J $\geq$ 3, there is one more step to perform: remove dominated regions. We use brute force's remove dominated regions algorithm for supernal method.

The extra complexity is: $2^n log\ 2^n + \frac{(J-1)(2^n-1)(2^n)}{2}$

So the total time complexity for the worst case is: $O(n) = (4^n)$

### 3.0 Instance Generation

The following algorithm generates an instance for the integer knapsack problem given number of items n, number of constraints m, number of objective functions J, and an Upper bound U for all integer objective function coefficients and Constraint coefficients U. Additionally, integer t is given which will be multiplied by the seed, to allow generating varying instances for a fixed n and U. This function returns list b containing m constraint right hand side, list c containing J objective function coefficient vectors , and list a that contains m constraint coefficient vectors. Note that, negative objective coefficients are generated to allow quick and direct calls to the Supernal, Rectangular and Bruteforce as these methods are solved based on minimization formulation.

```
Def Generate(n,m,J,U,t):

#Create an empty list to hold (minimization) objective function coefficients vectors
    c=[]
#Create an empty list for constraint rhs.
    b=[]
#Create an empty list for constraints coefficient vectors
    a_ik=[]
#Fix the seed
    last_five_digits= 54253*t
    seed=last_five_digits
    np.random.seed(seed)

#For each constraint, generate a coefficient vector neo, which contains a coefficient for all n elements. Append the
coefficient vectors to list a as they are created.
    for i in range(m) do
        neo->generate n random numbers each in range int(1,U+1)
        Append neo to a_ik
#Create a variable to hold the maximum coefficient in the constraint coefficient vector, initially set it to the first
coefficient in the vector. Iterate through all of the coefficients. In each iteration,  add the current coefficient to the
"summi" and compare the current coefficient to the "maxi". If it was larger, change maxi to current coef value.
        maxi=neo[0]
        summi=0
        for j in range(1,n) do
            cur=a_ik[i][j]
            summi+=cur
            if maxi<cur then
                maxi=cur
#For each constraint pick rhs as the maximum of maxi and [0.5 * Σ_{i=1}^{n} a_ik], Append it to the list b
        Append max(maxi,math.ceil(summi*0.5)) to b
#For each objective Z_j, generate an objective coefficient vector "new"  which contains  coefficient for all n
elements. Append the coefficient vectors to list c as they are created.
    for j in range(J) do
```

```
        new->generate n random numbers each in range int(-U-1,-1,n)
        Append new to c
    return b,c,a_ik
```

## 4.0 Algorithm Comparison

### 4.1 Comparison of Brute Force, Supernal and Rectangular - Varying n, m=fixed, J=2

It is aimed to compare the performance of all three methods for the same instances and plot them on a single figure to allow pairwise comparison. Function “test_performance_AVG” is implemented. [Appendix A]. “test_performance_AVG” function takes the maximum number of items to be generated during the instance generation. In addition, it takes a number of constraints m, number of objective functions J, and an Upper bound U. This function iterates through n=1, 2 , … , maxN. For each number of items n it generates 10 different instances, and tests all 10 instances using the three algorithms. The average run time of all ten instances with the same n is calculated for each algorithm and stored in a separate list. At the end the average run times are plotted against the corresponding number of items for all algorithms.

**Comparison of Brute Force, Supernal and Rectangular - varying n=1,2,...,18 m=2, J=2**
Figure A shows the graph that was generated through execution of “test_performance_AVG(U=90,MaxN=18,J=2,m=2)”. [Appendix A] To note, relatively small n was selected for maxN as BruteForce algorithm takes considerable amount of time.
Figure A suggests the complexity of all three algorithms are a function of n (number of items in the knapsack problem). Based on the graph,supernal performs the best out of the three methods for all instances. While the exact relation for Supernal and Rectangular is not clear, this graph suggests as the number of items increase brute force runtime exponentially increases which aligns with the complexity analysis result.

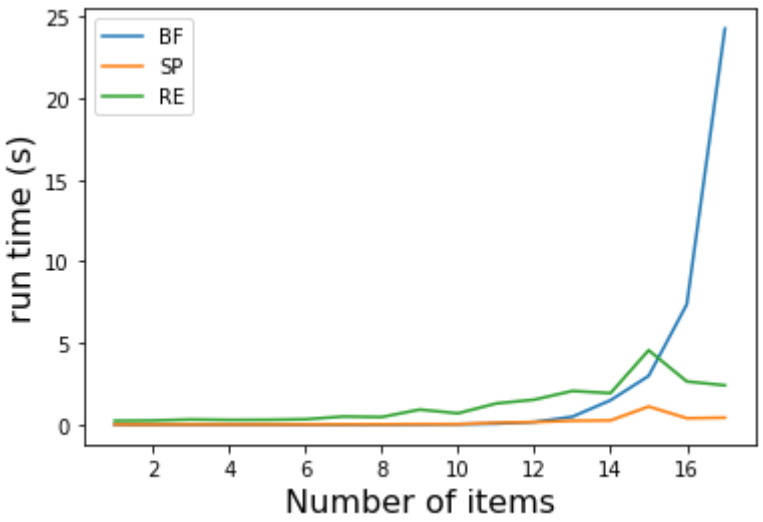


Figure A. Average run time vs. number of items (m=2, J=2)

**Rectangular performance and increase in the number of constraints**

Figure C shows the average run time for the Rectangular method when the number of items are 20 and number of objective functions is 2. For each m, currently 10 different points are generated and the average of run length is calculated for them. Additionally, the difficulty of different cases is only controlled by accepting the instance generations with above 1 second. Generating more instances and controlling the difficulty of cases will result in a more informing graph to identify the underlying relation between the complexity and m.

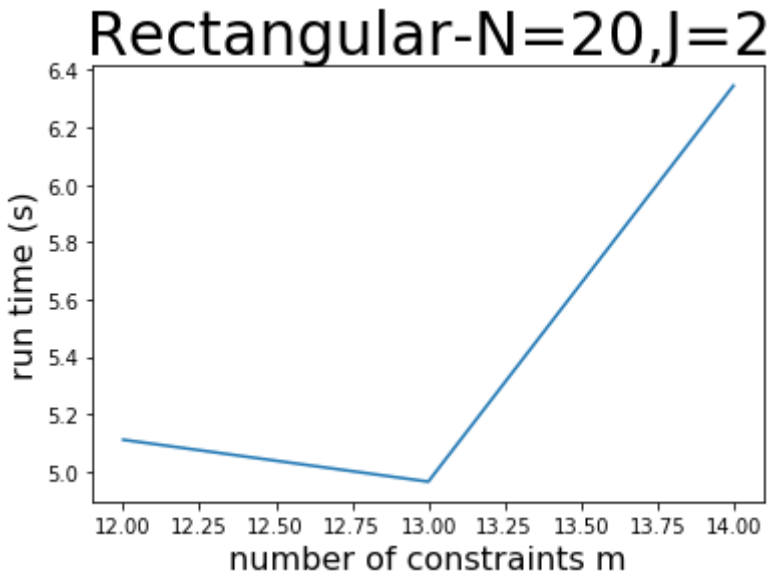


Figure B. Average run time vs. number of constraints - supernal

**Supernal Performance and increase in the number of constraints**

Figure C shows on average 0.3s increase when the number of constraints changes from 12 to 14. However, to investigate the relation between m and complexity, m should be increased further to obtain a better estimate of the function. [Appendix B]

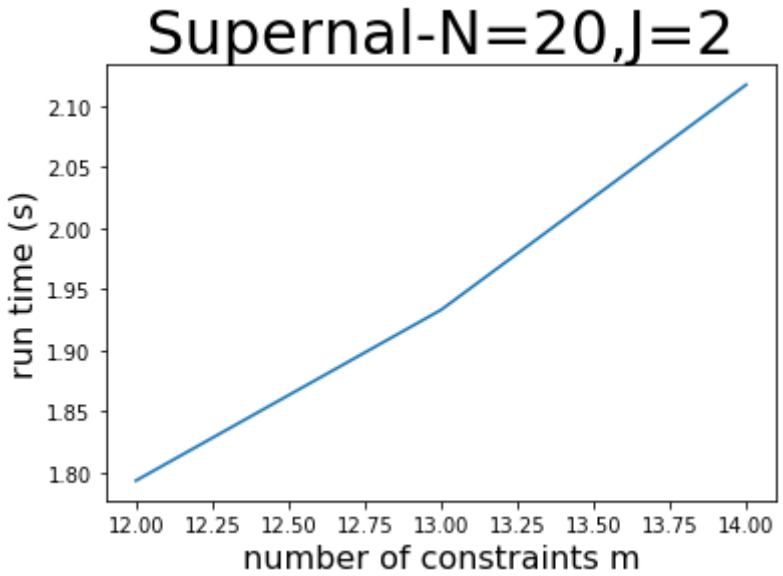


Figure C. Average run time vs. number of constraints - supernal

**Supernal Performance and varying J**

To investigate the relation between complexity and number of objective objective functions, number of items and constraints are fixed. Figure D shows the average runtime of 10 different instances for **J=3,4,5 when N=15 and m=2**. To note, the difficulty of the instance is checked by only keeping instances with larger than 1 second run time.

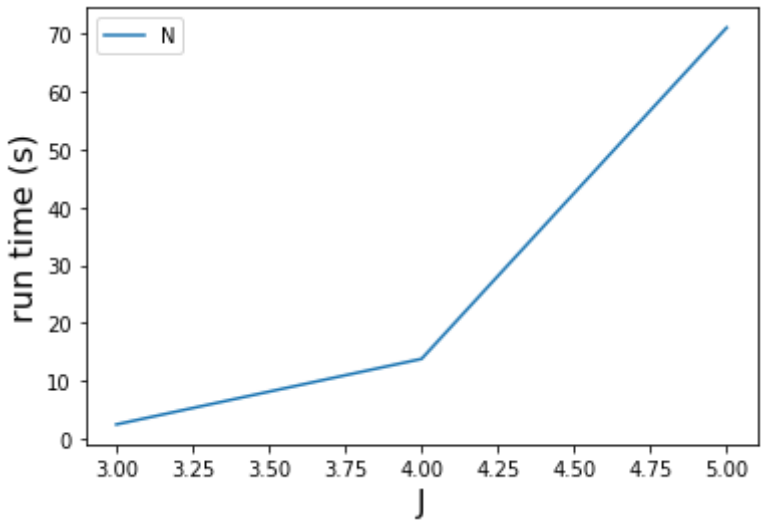


Figure D. Average run time vs. number of objective functions - supernal

**Comparison of Supernal Performance (varying N=10,20,30 and varying J = 3,4 and fixed m=2)**

Figure E suggests the average run time increase as both number of items and number of objective functions increase. Additionally, it could suggest that the complexity is a function of interaction of N and J in the supernal method.

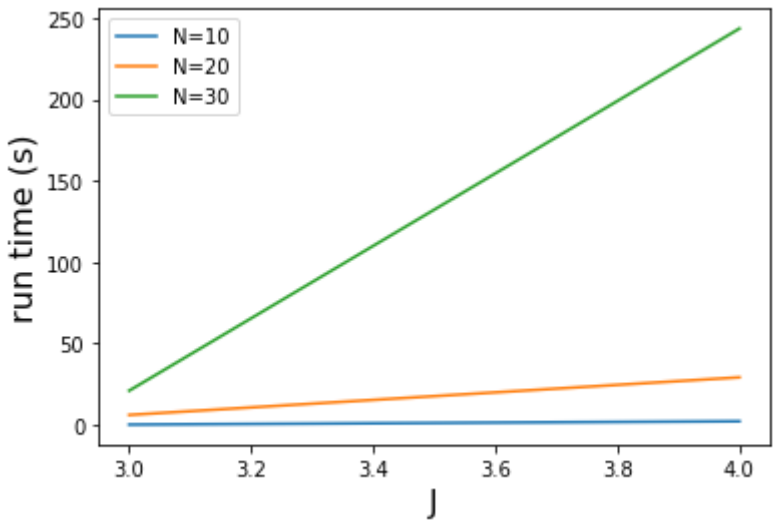


Figure E. Average run time vs. number of objective functions and items - supernal

**5.0 Memory-related issues**

The team has experienced memory issues on Brute-force function. When running this function with a large number of variables, the warning " Your startup disk is almost full " would pop out due to not enough space to store the generated permutations. Specifically, in our case, the problem is that all the iterables that 'itertools.product( )' produces are put in a list, resulting in all the combinations needing to exist at the same time. Therefore, it would consume too much RAM.

Here is the line of code: `lst = list(itertools.product([0, 1], repeat=n))`

This line of code shows that it does not iterate each item one by one, while doing work on each at the same time.

**6.0 Conclusion**

In this project, we code the baseline brute force method to prove how computationally expensive to solve a MOP in the decision space directly. After implementing the "divide-and-conquer" type of methods, we see significant improvements in both space and time complexity. Based on the results of algorithm comparisons, our supernal method is the most efficient for any number of objectives.

**References**

[1] "9.7. itertools - Functions creating iterators for efficient looping¶," *9.7. itertools - Functions creating iterators for efficient looping - Python 2.7.18rc1 documentation*. [Online]. Available: https://docs.python.org/2/library/itertools.html#itertools.product. [Accessed: 14-Apr-2020].

**Appendix**

A. Algorithm for test_performance_AVG(U,maxN,J,m)

```
#------testing and comparing
import matplotlib.pyplot as plt
def test_performance_AVG(U,maxN,J,m):
    runLength_BF=[]
    runLength_SP=[]
    runLength_RE=[]
    runLength_BF_AVG=[] # initialize an array to store the average run length of the ten instances with same number of items n solved using BF
    runLength_SP_AVG=[]
    runLength_RE_AVG=[]
    instance=[]
    for n in range(1,maxN):
        runLength_BF=[] #clear the array
        runLength_SP=[] #clear the array
        runLength_RE=[] #clear the array
        instance.append(n)
        for i in range(1,10):
             #generate an instance using Generate function with t being a random integer number in range(1,22) which is multiplied to the seed
            B,C,A=Generate(n,m,J,U,random.randint(1,22))
            t1=time.time() #set t1 to current time
            ndp1,ndpNum,exploNum=supernal(n,B,C,A,J)
            t1=time.time()-t1
            Append t1 to runLength_SP
            t1=time.time() #reset t1 to current time
            ndp,ndpNum,exploNum =BruteForce(n,B,C,A,J)
            t1=time.time()-t1
            Append t1 to runLength_BF
            t1=time.time() #reset t1 to current time
            ndp,ndpNum,exploNum=Rectangular(n,B,C,A,J)
            t1=time.time()-t1
            Append t1 to runLength_RE
        runLength_BF_AVG.append(np.mean(runLength_BF))
        runLength_SP_AVG.append(np.mean(runLength_SP))
        runLength_RE_AVG.append(np.mean(runLength_RE))

    plt.plot(instance,runLength_BF_AVG)
    plt.plot(instance,runLength_SP_AVG)
    plt.plot(instance,runLength_RE_AVG)
```

Appendix B

```
#------testing and comparing fixed j
import matplotlib.pyplot as plt
import random
import numpy as np
def test_rect_fixedJ_AVG(U,n,J,maxM):
```

```
    runLength_SP=[]
    runLength_SP_AVG=[]
    instance=[]
    t=1
    for m in range(12,maxM):
        runLength_SP=[]
        instance.append(m)
        for i in range(10):
            while(True):
                B,C,A=Generate(n,m,J,U,t)
                t1=time.time()
                ndp,ndpNum,exploNum=Rectangular(n,B,C,A,J)
                t1=time.time()-t1
                t+=1
                if t1>=1:
                    break
            runLength_SP.append(t1)


        runLength_SP_AVG.append(np.mean(runLength_SP))
    plt.plot(instance,runLength_SP_AVG)
    plt.xlabel('number of constraints m', fontsize=16);
    plt.ylabel('run time (s)' , fontsize=16);
    plt.title("Rectangular-N=20,J=2" ,fontsize=30)
    plt.show()

test_rect_fixedJ_AVG(90,20,2,15)
```